Pipeline de Certificados — ciszukoantony

Cómo funcionan los previews, cómo regenerarlos, subirlos al CDN y verificarlos.

Arquitectura

```
shared/docs/certificados/            ← fuente (PDFs/JPG, git-tracked)
shared/docs/certificados/previews/   ← previews generadas (JPG, git-tracked)
website/scripts/sync-certificates.js ← rasteriza PDFs → JPG + escribe el manifiesto
website/scripts/upload-previews.js   ← sube previews al bucket Supabase `ciszu-cdn`
website/src/data/certificates.previews.ts ← manifiesto AUTO-GENERADO (no editar a mano)
website/src/data/certificates.ts     ← metadatos de cada certificado (cards/modal)
website/src/app/certificates/page.tsx ← UI (cards, modal, filtros, logos de marca)
```

El sitio resuelve cada preview con `PREVIEWS_BY_FILE[mainFile.name]` y construye la URL `{CDN_BASE}/shared/docs/certificados/previews/{nombre}`. Si no hay mapping, la card muestra el estado "no preview" (nunca una imagen rota).

Reglas importantes

1. **Previews SIEMPRE en JPG** con nombre ASCII-safe: `sync-certificates.js` normaliza acentos (ó→o, ñ→n) porque **Supabase Storage rechaza claves con caracteres no-ASCII** (HTTP 400 `InvalidKey`). Los archivos fuente conservan su nombre original.
2. **Texto visible**: el render usa `standardFontDataUrl` + `cMapUrl` de `pdfjs-dist`. Sin eso, los PDFs con fuentes estándar (Helvetica, Times) renderizan **sin texto** — era el bug de las previews de SkillsBuild.
3. **Idempotente**: `sync` reutiliza previews existentes; usa `--force` para regenerar todo.
4. **El CDN es la fuente de verdad en producción**: si el bucket no tiene el archivo, la preview no carga. Verifica siempre tras subir.

Comandos

```
# 1) Regenerar previews (solo faltantes) + manifiesto
pnpm --filter ciszukoantony-website sync:certificates
#    ...o regenerar TODO desde cero:
node website/scripts/sync-certificates.js --force

# 2) Subir previews al CDN (solo las que faltan; --force re-sube todo)
node website/scripts/upload-previews.js

# 3) Verificar que cada preview mapeada responde 200 en el CDN
node website/scripts/upload-previews.js --verify-only
node website/scripts/verify-previews.js   # valida el manifiesto
completo end-to-end

# 4) (One-off) Logos de marca oficiales → CDN
node website/scripts/upload-brand-logos.js
```

Trampa conocida: `.vercelignore`

Tanto el `.vercelignore` del repo como el de `website/` tuvieron el patrón `**/downloads/**` (pensado para instaladores binarios que no existen). Eso impedía que Vercel recibiera `src/app/downloads/page.tsx`: la ruta nunca se compilaba y `/downloads` daba 404 en producción aunque el build local funcionara. Si una página desaparece del build de Vercel pero existe localmente, **revisa** `.vercelignore` y verifica la tabla `Route (app)` del build (API: `GET /v2/deployments/{id}/events`).

Verificación end-to-end (checklist)

- [ ] `sync:certificates` corre sin errores y el manifiesto tiene 1 entrada por documento.
- [ ] `upload-previews.js --verify-only` → `63/63 responden 200`.
- [ ] `verify-previews.js` → `0 sin preview mapeado` y `todos responden 200`.
- [ ] `npx tsc --noEmit` sin errores y `next build` exitoso.
- [ ] En la página: tag “Owned by FRANCISCO ANTONIO GARCIA MENOLASCINA” visible en cada card y en el modal; logos de marca renderizan (SVG oficial del CDN o wordmark); icono de error usa el triángulo de alerta estándar.

Logos de marca

Los SVG oficiales (simple-icons) de Cisco, IBM, Microsoft y HP viven en `{CDN}/assets/brand-logos/*.svg` y se renderizan con `<BrandLogo>`. Marcas sin SVG oficial disponible (EF SET, Penn, 16Personalities, Simplilearn) usan wordmarks tipográficos con su color corporativo — nunca emojis ni paths inventados.